



Département Génie Informatique
Faculté des Sciences et Techniques, Tanger
Université Abdelmalek Essaâdi, Maroc
Année Universitaire 2025/2026

Système Intelligent de Télésurveillance et Prédiction de Crises Sanitaires

Encadré par : Prof. Mohamed EL BRAK

Réalisé par : BAKAR AMAR Lareibiya, Attouf Sara, Lehoueime Yahfdhou, MINYURANO Aloys
IASD, S2 – Département Génie Informatique, FST de Tanger

Résumé

Ce travail présente la conception, le développement et l'évaluation d'un système complet de télésurveillance médicale intelligente, baptisé **SmartHealth**. L'objectif est d'assurer le suivi continu de patients à domicile en combinant les technologies de l'Internet des Objets (IoT) et de l'Intelligence Artificielle (IA), répondant ainsi aux défis posés par le vieillissement démographique et la surcharge des structures hospitalières.

Le dispositif embarqué, construit autour d'un microcontrôleur ESP32 simulé sous Wokwi, collecte en temps réel cinq paramètres vitaux et environnementaux — fréquence cardiaque (BPM), saturation en oxygène (SpO₂), température corporelle, humidité ambiante et détection de chute — via des capteurs biométriques (MAX30100, DHT22) et un accéléromètre MPU6050. Les données sont publiées au format JSON via le protocole MQTT (broker public EMQX) et ingérées par la plateforme ThingsBoard Cloud pour le stockage temporel, la visualisation temps réel et la gestion graduée des alarmes.

Un orchestrateur Node-RED assure le routage des flux de données, la logique de décision et l'invocation d'une API d'analyse d'anomalies. Cette API, développée en Python avec **FastAPI** (port 8000) et scikit-learn, implémente une approche hybride alliant un modèle Random Forest (98,05 % de précision sur 2 000 échantillons de test) à un moteur de règles médicales expertes structurant trois niveaux d'alerte : Stable, Stress et Critique. En cas d'événement critique, un bot Telegram déclenche une notification instantanée au soignant selon une hiérarchie de priorité médicale (chute > tachycardie critique > bradycardie > hypoxémie). Les résultats expérimentaux, validés sur cinq scénarios cliniques, démontrent une latence globale inférieure à trois secondes, satisfaisant pleinement les exigences de la surveillance médicale en temps réel.

Mots-clés : IoT médical, Internet of Medical Things (IoMT), Intelligence Artificielle, détection d'anomalies, Random Forest, classification multiclasse, MQTT, Node-RED, ThingsBoard, ESP32, télésurveillance médicale, e-santé, capteurs biométriques, SmartHealth.

I. Introduction

La télésurveillance médicale constitue aujourd'hui un pilier fondamental de la transformation numérique des systèmes de santé, particulièrement dans le contexte du vieillissement démographique mondial. Selon les projections de l'Organisation Mondiale de la Santé

(OMS), la proportion de la population âgée de plus de 60 ans devrait doubler d'ici 2050, portant à plus de deux milliards le nombre de personnes âgées à l'échelle mondiale [1]. Cette évolution démographique exerce une pression croissante sur les structures hospitalières et rend indispensable le développement de solutions permettant le maintien à domicile des patients tout en assurant leur

sécurité physiologique.

Parallèlement, la récente crise sanitaire mondiale a mis en lumière les limites des systèmes de santé traditionnels face aux pics de demande et a accéléré l'adoption des technologies de télémédecine. Dans ce contexte, l'*Internet of Medical Things* (IoMT) — déclinaison médicale de l'Internet des Objets — offre des perspectives inédites en connectant capteurs biométriques, passerelles réseau et plateformes cloud au service du patient. L'association synergique de l'IoT et de l'Intelligence Artificielle (IA) permet de dépasser les simples systèmes de transmission de données pour atteindre un niveau d'analyse prédictive et préventive, transformant un objet connecté en véritable dispositif médical intelligent.

Le projet **SmartHealth** s'inscrit dans cette dynamique et vise à développer un prototype fonctionnel de télésurveillance intelligente répondant à cinq objectifs :

1. Acquisition fiable en temps réel de cinq paramètres vitaux et environnementaux via des capteurs simulés sur microcontrôleur ESP32.
2. Transmission sécurisée et légère des données vers le cloud via le protocole MQTT, adapté aux contraintes de l'IoT médical.
3. Orchestration des flux de données et logique de décision automatisée grâce à Node-RED.
4. Classification hybride des états patients (Stable, Stress, Critique) par un modèle Random Forest couplé à un moteur de règles médicales expertes.
5. Génération automatisée d'alertes graduées avec notification instantanée au soignant via Telegram, selon une hiérarchie de priorité médicale.

Ce rapport est organisé comme suit : la Section II présente un état de l'art des technologies IoMT et des approches IA associées ; la Section III décrit l'architecture globale du système ; les Sections IV et V détaillent respectivement la couche IoT et le module d'Intelligence Artificielle ; la Section VI présente les résultats expérimentaux ; la Section VII discute les performances, les limites et les perspectives d'amélioration.

II. Travaux Connexes

L'intégration de l'IoT dans le domaine médical (IoMT) a fait l'objet de nombreux travaux. Baker et al. [2] proposent une taxonomie des architectures IoT médicales distinguant trois couches : perception, réseau, et application. Parmi les protocoles de transport, MQTT s'est imposé comme standard de fait grâce à son modèle publish/subscribe léger [3].

Concernant les plateformes cloud, ThingsBoard se distingue par son modèle open-source et ses capacités de dashboarding temps réel. Abaya et al. [4] l'utilisent pour la surveillance de patients cardiaques. Node-RED, framework de programmation visuelle développé

par IBM, est fréquemment employé comme couche d'orchestration [7].

Du côté IA, Breiman [5] a introduit les forêts aléatoires comme algorithme robuste de classification multiclass. Liu et al. [6] ont proposé l'Isolation Forest, particulièrement adapté à la détection d'anomalies en temps réel. La littérature récente montre l'efficacité des approches hybrides combinant apprentissage automatique et règles expertes pour la surveillance médicale [8].

III. Architecture Globale du Système

L'architecture du système SmartHealth adopte une organisation en quatre couches fonctionnelles :

1. **Couche Perception** — Capteurs biométriques (simulés), accéléromètre MPU6050 et microcontrôleur ESP32.
2. **Couche Transport** — Communication MQTT via broker public (broker.emqx.io) sur le port 1883.
3. **Couche Orchestration** — Flux Node-RED pour le routage, l'accumulation des données et l'invocation des services.
4. **Couche Intelligence & Visualisation** — API Python (FastAPI + scikit-learn) et dashboard ThingsBoard Cloud.

Le flux opérationnel du système suit le schéma suivant : l'ESP32 lit les capteurs toutes les 8 secondes et publie un objet JSON sur le topic `health/patient1/sensors`. Node-RED achemine les données vers l'API IA et vers ThingsBoard. En cas de sévérité *Critique*, une alerte Telegram est déclenchée immédiatement.

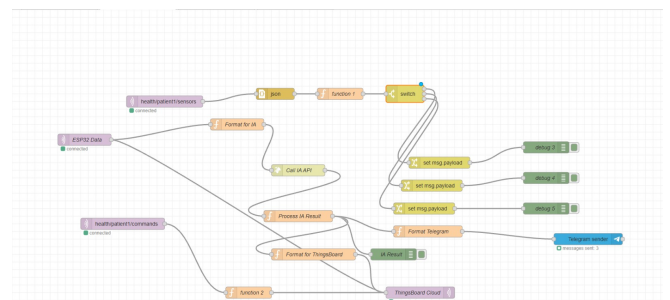


Figure 1: Flow complet Node-RED : réception MQTT → traitement IA → ThingsBoard + Telegram

A. Couche Perception : ESP32 et Capteurs

Le dispositif embarqué repose sur le microcontrôleur ESP32 simulé dans l'environnement Wokwi. Le firmware Arduino intègre les bibliothèques `WiFi.h`, `PubSubClient.h`, `ArduinoJson.h`, `DHT.h` et `Adafruit MPU6050.h`. Le Tableau 1 récapitule les capteurs déployés.

Table 1: Capteurs et paramètres mesurés

Paramètre	Capteur	Plage normale / Seuil
BPM	PPG simulé	60–100 bpm
SpO ₂	MAX30100	95–100 %
Température	DHT22 (GPIO 15)	36–37,5 °C
Humidité	DHT22 (GPIO 15)	40–60 %
Chute	MPU6050 (3 axes)	Détection si accélération

```

Initialisation DHT22... [x]
Initialisation MPU6050... [x] (Mode simulation)
Connexion WiFi..... [x]
IP : 10.10.0.2
Connexion MQTT... [x] Connecte !
Abonne au topic: health/patient1/commands

[x] Systeme pret !
[x] MQTT Publish: health/patient1/sensors
[x] MQTT Subscribe: health/patient1/commands
[x] Envoi toutes les 8 secondes
[x] LED clignote toutes les 5 secondes si MQTT connecte

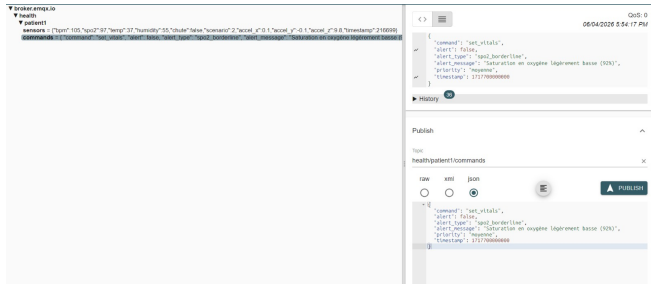
=====
Scenario 1/5 : NORMAL
BPM=75 | SpO2=95% | Temp=37.0°C | Chute=NON
Payload:
{"bpm":75,"spo2":95,"temp":37,"humidity":55,"chute":false,"scenario":1,"accel_x":0.1,"accel_y":0.1,"accel_z":9.8,"timestamp":8843}
[x] MQTT publie avec succes !
=====

```

Figure 2: Simulation Wokwi : initialisation DHT22/MPU6050, connexion WiFi+MQTT, publication JSON toutes les 8 secondes avec confirmation OK

B. Communication MQTT

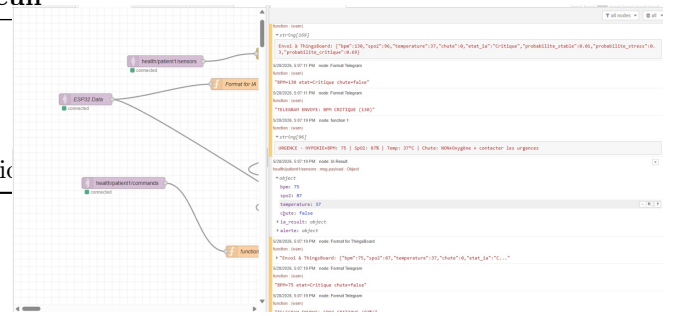
Le choix du protocole MQTT s'est imposé compte tenu des contraintes de l'IoT médical : bande passante limitée, instabilité potentielle du WiFi, nécessité d'une faible latence. MQTT repose sur un modèle publish/subscribe où les producteurs publient des messages sur des topics hiérarchiques sans connaissance directe des consommateurs [3].

**Figure 3:** MQTT Explorer : broker `broker.emqx.io`, topic `health/patient1/commands` avec payload JSON détaillé (`set_vitals`, `spo2_borderline`, priorité moyenne). Cette capture illustre la **communication bidirectionnelle** du système SmartHealth : le broker EMQX reçoit et transmet une commande contenant le résultat de l'analyse IA vers le dispositif ESP32, confirmant que la boucle cloud → capteur fonctionne correctement en complément du flux capteur → cloud.

C. Orchestration Node-RED

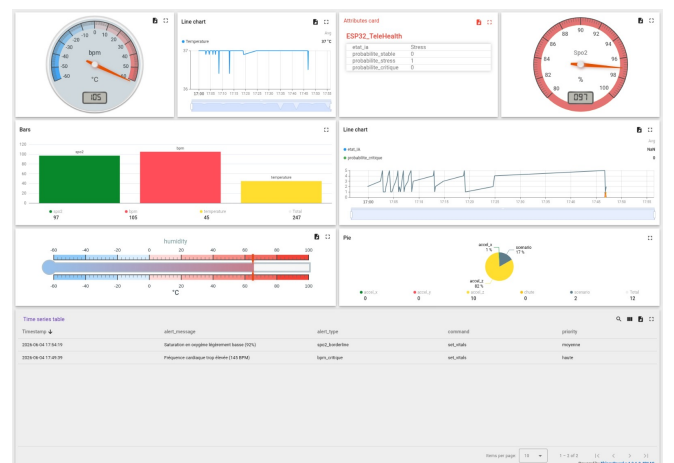
Node-RED constitue la colonne vertébrale opérationnelle du système. L'éditeur graphique permet de concevoir visuellement le flux sans code impératif complexe. Le nœud d'entrée souscrit au topic de télémétrie, puis distribue chaque message vers : envoi ThingsBoard, appel

API IA, et vérification immédiate des seuils critiques via un nœud Switch

**Figure 4:** Débogage Node-RED : résultat IA (Critique, probabilité=0,69), formatage Telegram, envoi ThingsBoard avec BPM=130

D. Plateforme Cloud ThingsBoard

ThingsBoard Cloud assure la gestion du device, la persistance temporelle et la visualisation interactive. Le dashboard présente des jauges circulaires (BPM, SpO₂, Température), un graphique temporel mettant en évidence les épisodes anormaux, un diagramme circulaire et un histogramme historique.

**Figure 5:** Dashboard ThingsBoard ESP32_TeleHealth : jauges temps réel (BPM=75, SpO₂=97 %), état IA=Critique, tableau d'alarmes actives

E. Persistance ThingsBoard des Alertes

La plateforme ThingsBoard assure non seulement la visualisation temps réel des données physiologiques, mais aussi la persistance historique de toutes les télémétries et événements d'alerte. Cette fonctionnalité est essentielle pour la traçabilité médicale : chaque prédiction du modèle IA, chaque commande générée et chaque changement d'état du patient est enregistré dans une base de données temporelle (time-series). Les figures suivantes illustrent cette capacité de persistance à travers l'onglet *Latest Telemetry* qui expose les 25 dernières clés de télémétrie du device, et la *Time Series Table* qui présente l'historique chronologique des alertes avec leur niveau de priorité associé.

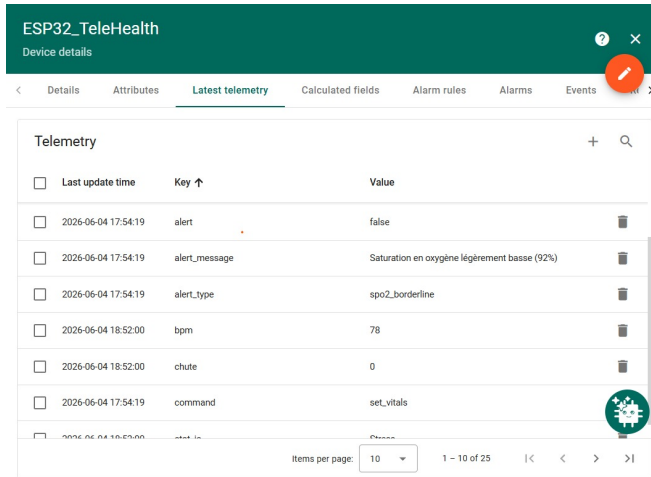


Figure 6: Onglet *Latest Telemetry* du device ESP32_TeleHealth sur ThingsBoard (25 clés de télémétrie).

On y retrouve à la fois les **données physiologiques brutes** (bpm=78, chute=0) et les **métadonnées d'analyse IA** (alert=false, alert_message="Saturation en oxygène légèrement basse (92%)", alert_type=spo2_borderline, command=set_vitals). La coexistence de ces deux catégories dans le même device confirme l'intégration complète du pipeline : les résultats du modèle Random Forest sont persistés aux côtés des données capteurs, offrant une traçabilité médicale complète.



Figure 7: Time Series Table ThingsBoard — Historique des alertes médicales persistées. La première entrée (17:54:19) correspond à une saturation en oxygène légèrement basse (92%) — type **spo2_borderline**, priorité **moyenne**. La seconde (17:49:39) signale une fréquence cardiaque trop élevée à 145 BPM — type **bpm_critique**, priorité **haute**. Ces enregistrements démontrent la persistance temporelle des événements cliniques et la graduation correcte des niveaux de priorité par le moteur de règles expertes du système.



Figure 8: Vue de la colonne **priority** dans la Time Series Table de ThingsBoard. L'enregistrement du 04/06/2026 à 17:54:19 affiche une priorité **moyenne** pour l'alerte **spo2_borderline**. Cette colonne confirme que le système de graduation des alertes fonctionne correctement : chaque commande reçue est non seulement persistée en base de données temporelle, mais aussi enrichie de son niveau de criticité, permettant au soignant de filtrer et prioriser efficacement les événements médicaux.

F. Alerte Telegram

Un bot Telegram envoie des notifications formatées dès la détection d'un événement critique, incluant : type d'alerte, valeurs physiologiques instantanées, état IA et recommandation d'action.

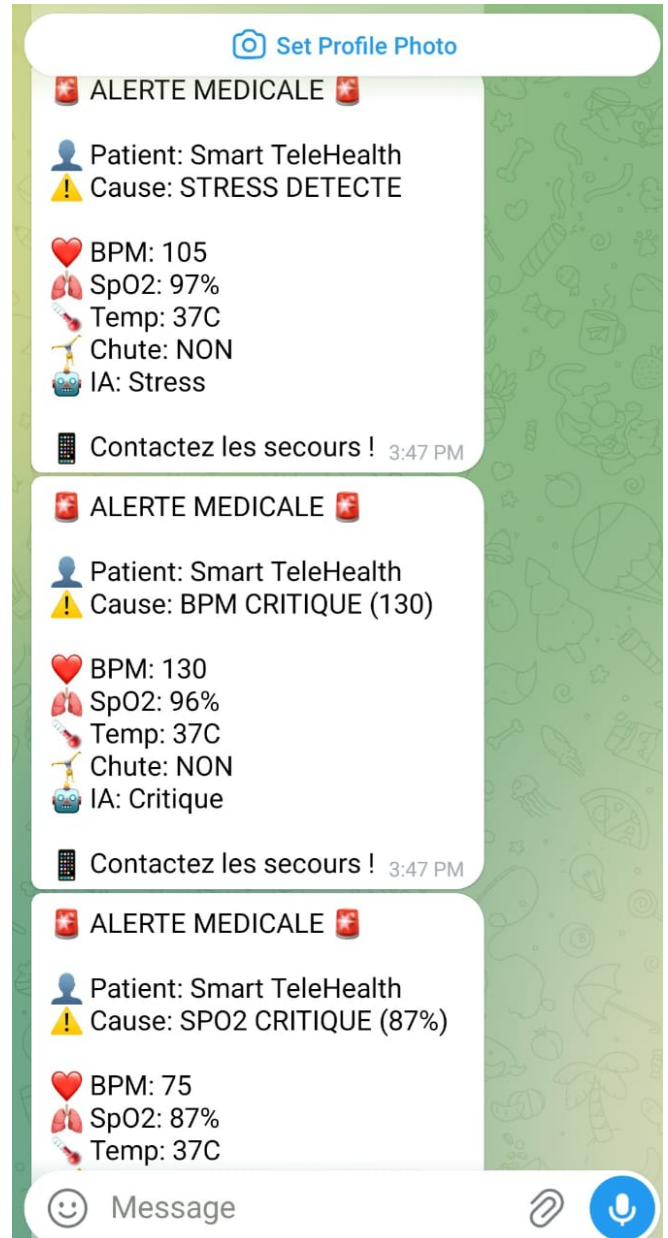


Figure 9: Notifications Telegram : STRESS DÉTECTÉ (BPM=105), BPM CRITIQUE (130), SpO₂ CRITIQUE (87%)

IV. Module d'Intelligence Artificielle

A. Génération du Dataset Médical

Un dataset médical réaliste de 10 000 échantillons a été généré synthétiquement via `generate_dataset.py`. Les features incluses dans le modèle sont les suivantes : BPM, SpO₂, température, âge, comorbidités, médicaments, pression artérielle (sys/dia), fréquence respiratoire (RR) et détection de chute. **L'humidité ambiante, bien que collectée par le capteur DHT22, n'est pas utilisée comme feature d'entrée du modèle Random Forest, car ce paramètre environnemental n'est pas cliniquement pertinent pour la classification de l'état physiologique du patient**

(Stable/Stress/Critique).

Aperçu des données (10 premières lignes)

	bpm	sp02	temperature	chute	age	comorbidity_count	medication_count	bp_sys	bp_dia	rr	label
0	94	94	37.4	0	65	2	2	111	73	19	1
1	72	94	37.0	0	41	0	0	112	79	14	1
2	66	98	37.3	0	58	0	0	105	74	16	0
3	106	92	37.9	0	77	1	1	136	83	20	1
4	119	92	37.3	0	18	0	0	133	87	18	1
5	78	96	36.2	0	80	0	0	111	75	16	1
6	71	96	36.7	0	88	1	0	120	67	15	0
7	111	93	37.6	0	21	0	0	139	73	17	1
8	69	97	36.3	0	36	2	1	116	82	13	1
9	72	95	36.7	0	30	2	1	106	73	16	0

Distribution finale des classes

Stable (0):	3334 samples (33.3%)
Stress (1):	3333 samples (33.3%)
Critique (2):	3333 samples (33.3%)

Figure 10: Aperçu des dix premières lignes du dataset généré. Les colonnes représentent les paramètres cliniques (bpm, sp02, température, chute, âge, comorbidités, médicaments, pression artérielle, fréquence respiratoire) et l'étiquette de classification (0=Stable, 1=Stress, 2=Critique).

La figure 10 présente un aperçu des dix premières lignes du dataset généré. Chaque ligne correspond à un enregistrement patient contenant les dix variables cliniques. Cet aperçu illustre la variabilité réaliste des paramètres générés, avec des valeurs de BPM allant de 66 à 119 bpm, des saturations en oxygène comprises entre 92% et 98%, et des températures variant entre 36,2 et 37,9 °C.

EXPLORER

- IA
 - .venv
 - api.py
 - curl.exe
 - generate_dataset.py
 - model_realistic.joblib
 - patient_data_realistic.csv
 - train_model.py
 - training_metrics.json
 - training_metrics.txt

Figure 11: Distribution finale des trois classes après équilibrage. Le dataset contient 3334 échantillons pour la classe Stable (33,3%) et 3333 échantillons pour les classes Stress et Critique (33,3% chacune).

La figure 11 montre la distribution finale des trois classes après application des techniques d'équilibrage (undersampling et oversampling). Le dataset final est parfaitement équilibré avec 33,3% des échantillons par classe. Cet équilibrage est essentiel pour éviter les biais d'apprentissage et garantir que le modèle Random Forest ne favorise aucune classe particulière.

Un équilibrage par undersampling et oversampling aboutit à 3333 échantillons par classe.

Génération du dataset médical

nombre d'échantillons: 10000

Génération des données...

Distribution des classes

Stable (0):	3515 samples (35.1%)
Stress (1):	4281 samples (42.8%)
Critique (2):	2204 samples (22.0%)

Équilibrage exact des classes

Undersampling: réduire de 181 échantillons pour Stable (0)

Undersampling: réduire de 948 échantillons pour Stress (1)

Oversampling: ajouter 1129 échantillons pour Critique (2)

Statistiques finales

Total d'échantillons: 10000

Colonnes: ['bpm', 'sp02', 'temperature', 'chute', 'age', 'comorbidity_count', 'medication_count', 'bp_sys', 'bp_dia', 'rr', 'label']

BPM: min=50, max=170, mean=98.1

sp02: min=70, max=100, mean=93.0

température: min=35.6, max=41.0, mean=37.4

Dataset sauvegardé: patient_data_realistic.csv

Taille: 10000 lignes

Figure 12: Génération du dataset : 10 000 échantillons, distribution initiale (Stable 35,1 %, Stress 42,8 %, Critique 22,0%), puis équilibrage exact

B. Entraînement Random Forest

Le modèle Random Forest est entraîné via `train_model.py` avec un split 80/20. Le Tableau 2 présente les performances obtenues sur l'ensemble de test (2 000 échantillons).

Table 2: Performances du modèle Random Forest

Classe	Précision	Recall	F1	Support
Stable (0)	0,99	0,97	0,98	667
Stress (1)	0,96	0,99	0,97	667
Critique (2)	0,99	0,98	0,99	666
Weighted Avg	0,98	0,98	0,98	2 000

```

PS C:\Users\HP\Downloads\IA> c:\Users\HP\Downloads\IA\.venv\scripts\python.exe train_model.py patient_data_realistic.csv
v model_realistic.joblib
loading data from patient_data_realistic.csv...
splitting train/test...
Training model...
Evaluating...

```

	precision	recall	f1-score	support
0	0.99	0.97	0.98	667
1	0.96	0.99	0.97	667
2	0.99	0.98	0.99	666
accuracy			0.98	2000
macro avg	0.98	0.98	0.98	2000
weighted avg	0.98	0.98	0.98	2000

```

Accuracy: 0.9805
Confusion matrix:
[[645 20  2]
 [ 4 660  3]
 [ 3  7 656]]
Saved trained model to model_realistic.joblib
PS C:\Users\HP\Downloads\IA>

```

Figure 13: Terminal d'entraînement : rapport de classification et matrice de confusion. Accuracy globale = 98,05 % sur 2 000 échantillons de test

C. API de Prédiction (FastAPI)

Le modèle est exposé via une API REST FastAPI (`api.py`) sur le port 8000 (port par défaut du framework FastAPI). Node-RED envoie les données du patient en JSON vers `http://127.0.0.1:8000/predict` et reçoit l'état IA, les probabilités par classe et les recommandations médicales.

V. Tests, Validation et Résultats

A. Scénarios de Test

La validation repose sur cinq scénarios cliniques simulés. Le Tableau 3 récapitule pour chaque scénario les paramètres d'entrée, la prédiction du modèle IA, l'alerte générée et la latence mesurée.

Table 3: Récapitulatif des cinq scénarios de test cliniques

Scénario	Entrée (BPM/SpO/Temp/Chute)	Prédic
1 - Normal	73 / 95% / 36.8 / Non	Stable
2 - Stress	105 / 96% / 37.1 / Non	Stress
3 - BPM Critique	130 / 94% / 37.3 / Non	Critique
4 - SpO Critique	82 / 87% / 37.0 / Non	Critique
5 - Chute	90 / 96% / 36.9 / Oui	Critique

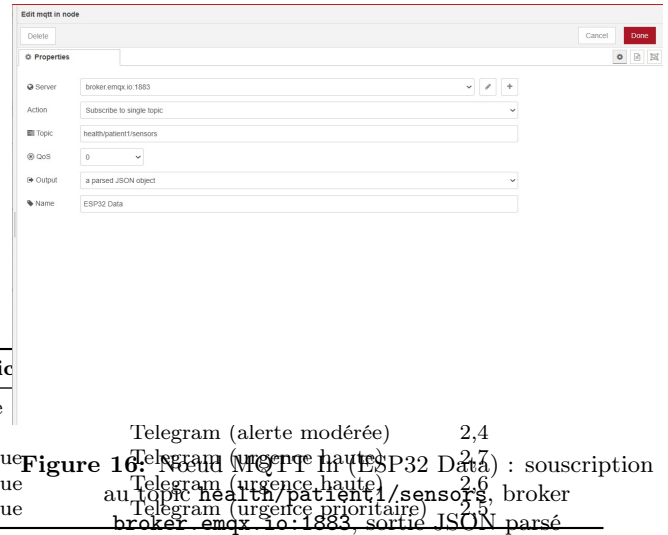


Figure 16: Nœud MQTT In (ESP32 Data) : souscription au topic `health/patient1/sensors`, broker `broker.emqx.io:1883`, sortie JSON parsé

Les Figures 14 et 15 montrent les payloads publiés et confirmés sur MQTT pour chaque scénario.

```
Scenario 1/S : NORMAL
BPM=73 | SpO2=95% | Temp=37.0°C | Chute=NON
Payload:
{"bpm":73,"spo2":95,"temp":37,"humidity":55,"chute":false,"scenario":1,"accel_x":0.1,"accel_y":-0.1,"accel_z":9.8,"timestamp":8847}
MQTT publié avec succès !

Scenario 2/S : BPM ÉLEVÉ
BPM=105 | SpO2=97% | Temp=37.0°C | Chute=NON
Payload:
{"bpm":105,"spo2":97,"temp":37,"humidity":55,"chute":false,"scenario":2,"accel_x":0.1,"accel_y":-0.1,"accel_z":9.8,"timestamp":16855}
MQTT publié avec succès !
```

Figure 14: Scénarios 1 (NORMAL : BPM=73, SpO₂=95 %) et 2 (BPM ÉLEVÉ : BPM=105). Publication MQTT confirmée

```
Scenario 3/S : BPM CRITIQUE
BPM=130 | SpO2=96% | Temp=37.0°C | Chute=NON
Payload:
{"bpm":130,"spo2":96,"temp":37,"humidity":55,"chute":false,"scenario":3,"accel_x":0.1,"accel_y":-0.1,"accel_z":9.8,"timestamp":24856}
MQTT publié avec succès !

Scenario 4/S : SpO2 CRITIQUE
BPM=75 | SpO2=87% | Temp=37.0°C | Chute=NON
Payload:
{"bpm":75,"spo2":87,"temp":37,"humidity":55,"chute":false,"scenario":4,"accel_x":0.1,"accel_y":-0.1,"accel_z":9.8,"timestamp":32864}
MQTT publié avec succès !

Scenario 5/S : CHUTE DETECTEE
BPM=90 | SpO2=97% | Temp=37.0°C | Chute=OUI
Payload:
{"bpm":90,"spo2":97,"temp":37,"humidity":55,"chute":true,"scenario":5,"accel_x":8.5,"accel_y":6.2,"accel_z":2.1,"timestamp":...}
```

Figure 15: Scénarios 3 (BPM CRITIQUE=130), 4 (SpO₂=87 %) et 5 (CHUTE : magnitude 10,6 g)

B. Configuration Détaillée des Nœuds Node-RED

Les figures suivantes détaillent la configuration individuelle de chaque nœud constituant le flux Node-RED du système SmartHealth.

Interprétation : Ce nœud est la **porte d'entrée absolue** du système SmartHealth dans Node-RED. Tout commence ici : c'est lui qui écoute en permanence le réseau et capte les données physiologiques dès qu'elles arrivent de l'ESP32.

Ce nœud maintient une connexion permanente ouverte avec le broker MQTT public EMQX hébergé sur

`broker.emqx.io`, port 1883. Il se comporte comme un abonné silencieux qui attend — et dès qu'un message arrive sur le topic `health/patient1/sensors`, il le capte immédiatement et déclenche l'ensemble du flux de traitement en aval.

Le topic `health/patient1/sensors` mérite une attention particulière. Sa structure hiérarchique en trois niveaux (domaine / identifiant patient / type de données) n'est pas anodine : elle révèle une conception pensée pour évoluer. Ajouter un deuxième patient revient simplement à créer un nouveau topic `health/patient2/sensors` sans modifier l'architecture, démontrant un principe d'extensibilité fondamental dans les systèmes IoT.

Le choix du QoS 0 est une décision d'ingénierie judicieuse. L'ESP32 publie de nouvelles mesures toutes les 8 secondes, ce qui rend la perte occasionnelle d'un message sans impact clinique significatif. Ce niveau de qualité de service permet d'optimiser les ressources réseau et la consommation énergétique de l'appareil embarqué.

L'option « a parsed JSON object » est cruciale pour la fluidité du flux. Node-RED déserialise automatiquement le JSON, rendant les champs `bpm`, `spo2`, `temp`, `chute`, `timestamp` directement accessibles comme propriétés d'un objet JavaScript, ce qui simplifie radicalement le code des nœuds Fonction suivants.

Interprétation : Ce nœud résout un problème architectural élégant : **comment envoyer uniquement ce dont l'IA a besoin, sans perdre les données originales dont le reste du système a besoin ?**

L'API IA attend un payload minimal et précis — uniquement `bpm`, `spo2`, `temperature`, `chute`. La solution implémentée est un mécanisme de sauvegarde/restauration en deux temps. Ce nœud copie l'intégralité du message original dans `msg.originalData` — un champ personnalisé qui voyage avec le message tout au long du flux, invisible pour l'IA mais récupérable par tous les autres nœuds.

La ligne `msg.payload.temp || msg.payload.temperature` gère les évolutions potentielles du firmware ESP32, acceptant indifféremment les deux noms de champ (`temp` ou `temperature`), ce qui démontre une pratique de robustesse pragmatique.

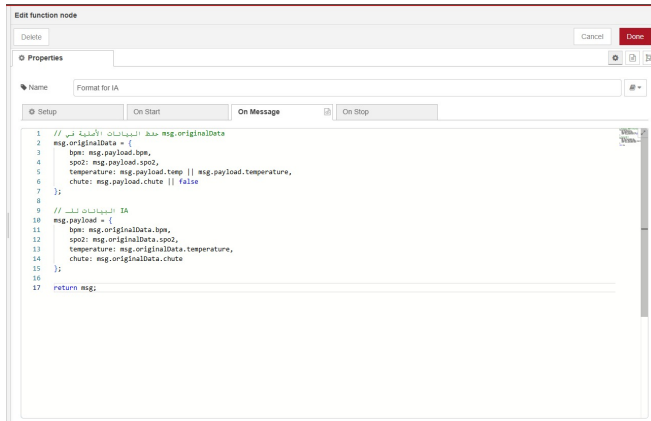


Figure 17: Nœud Fonction « Format for IA » : sauvegarde des données originales et formatage du payload pour l'API IA

La valeur par défaut `chute || false` protège le modèle IA contre les messages incomplets, injectant `false` plutôt que `undefined` pour éviter une prédiction erronée.

Ce nœud illustre parfaitement le rôle de Node-RED : une orchestration propre des données entre des services aux interfaces différentes, agissant comme traducteur entre le langage de l'ESP32 et le langage de l'IA.

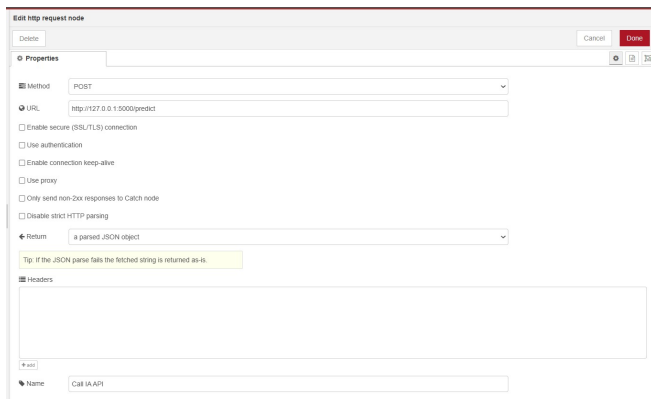


Figure 18: Nœud HTTP Request « Call IA API » : méthode POST vers `http://127.0.0.1:8000/predict`, retour JSON parsé

Interprétation : Ce nœud est le pont entre le monde de l'IoT et le monde de l'Intelligence Artificielle. Le flux Node-RED quitte son périmètre pour interroger l'API FastAPI qui héberge le modèle Random Forest entraîné.

Ce nœud prend `msg.payload` (les quatre paramètres vitaux formatés par le nœud précédent), l'envoie en POST à `http://127.0.0.1:8000/predict`, attend la réponse, la désérise automatiquement en JSON, et la place dans `msg.payload` pour les nœuds suivants, le tout en moins de 500 ms.

L'URL `127.0.0.1:8000` indique que Node-RED et l'API Python FastAPI s'exécutent sur la même machine physique, un choix d'architecture qui maximise la performance (pas de latence réseau) et simplifie le déploiement en développement.

Le retour « a parsed JSON object » rend la réponse de l'API — comme `{"etat_ia": "Critique",`

`"proba_stable": 0.01, "proba_stress": 0.30, "proba_critique": 0.69}` — directement exploitable dans le nœud suivant sans manipulation supplémentaire.

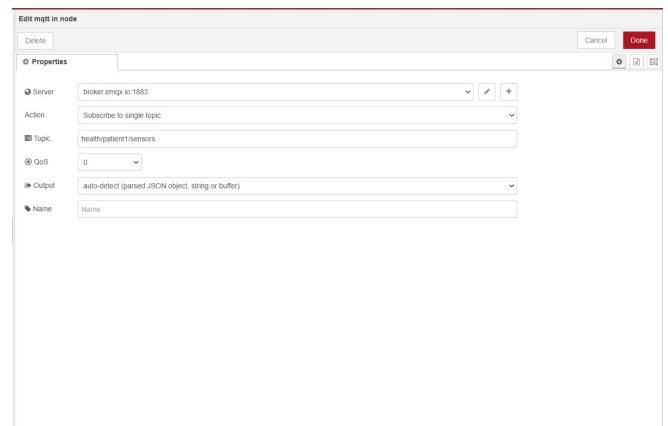


Figure 19: Nœud Fonction « Format for ThingsBoard » : construction du payload telemetry incluant état IA, probabilités et données vitales

Interprétation : Ce nœud est l'assembleur final qui réunit en un seul objet cohérent les données physiologiques brutes de l'ESP32 et l'intelligence produite par le modèle IA. C'est le payload qui alimente le dashboard visuel médical sur ThingsBoard.

Le code utilise un alias de compatibilité `msg.payload.temp || msg.payload.temperature` pour gérer indépendamment les variations potentielles du firmware.

La récupération du résultat IA utilise une triple tentative : `msg.payload.result || msg.payload.ia_result || msg.payload`, permettant au nœud de fonctionner quel que soit son emplacement dans le flow (avant ou après « Process IA Result »).

Le mapping `{0: "Stable", 1: "Stress", 2: "Critique"}` transforme les labels numériques en concepts médicaux explicites. La construction `chute ? 1 : 0` convertit le booléen en entier, optimisé pour les time-series de ThingsBoard.

Interprétation : Ce nœud est la sortie vers le cloud médical. Les données enrichies quittent le système local pour être persistées, historisées et visualisées sur le dashboard ThingsBoard.

Le serveur `mqtt.thingsboard.cloud:1883` est l'endpoint MQTT officiel de ThingsBoard Cloud. ThingsBoard comprend nativement la structure des messages : chaque clé JSON devient automatiquement une métrique dans la time-series du device.

Le topic `v1/devices/me/telemetry` est l'API officielle de ThingsBoard pour l'ingestion de télémétrie. Publier sur ce topic déclenche automatiquement la persistance des données, leur apparition dans les graphiques historiques et le déclenchement des règles d'alarme.

Le QoS 1 (At least once) garantit que les données arrivent bien et sont persistées dans le cloud, un choix prudent pour les données médicales.

Interprétation : Ce nœud représente la communication de retour, du cloud vers le dispositif.

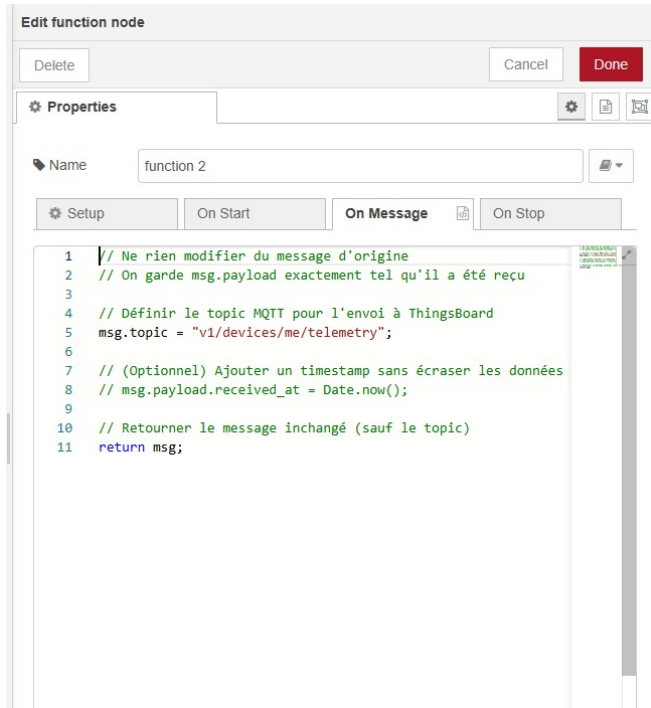


Figure 20: Nœud Fonction « function 2 » — Définition du topic MQTT pour ThingsBoard. Ce nœud assure le routage correct des données en définissant explicitement `msg.topic = "v1/devices/me/telemetry"` sans modifier le payload original (`msg.payload`). Ce mécanisme garantit que ThingsBoard reçoit les données sur son endpoint officiel d'ingestion de télémétrie, déclenchant automatiquement la persistance et la mise à jour des graphiques temps réel.

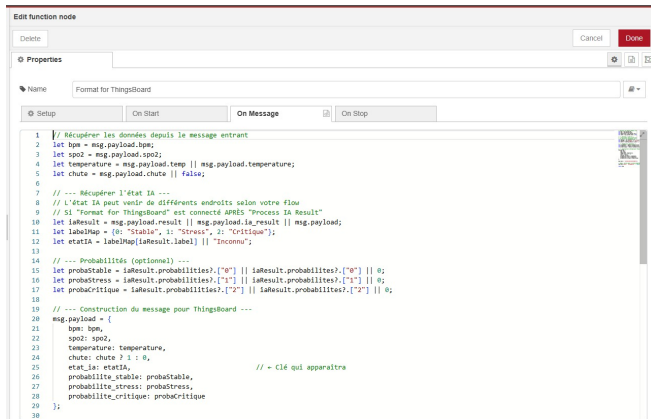


Figure 21: Nœud MQTT Out « ThingsBoard Cloud » : publication sur `mqtt.thingsboard.cloud:1883`, topic `v1/devices/me/telemetry`, QoS 1

SmartHealth n'est pas un système unidirectionnel — il peut aussi recevoir des instructions.

Le topic `health/patient1/commands` est le symétrique de `health/patient1/sensors`, révélant une architecture de messagerie bilatérale cohérente. Cette séparation claire des flux évite tout mélange entre données de mesure et données de contrôle.

Le QoS 2 (Exactly once) est judicieusement choisi pour les commandes médicales. Une commande comme « `reset_alarm` » doit être exécutée exactement une fois — ni zéro, ni deux. Le protocole d'acquiescement en quatre

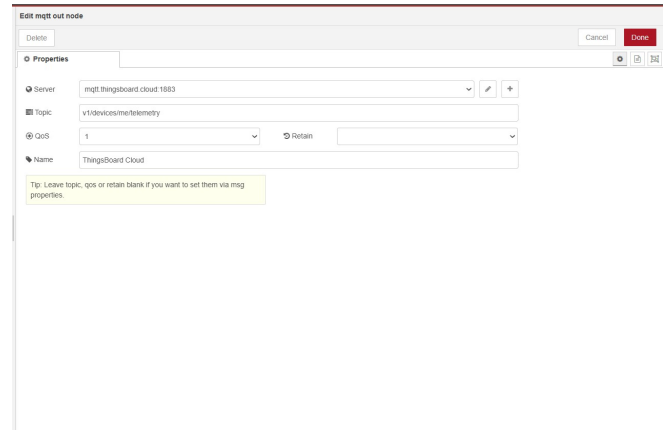


Figure 22: Nœud MQTT In (topic commands) : réception des commandes depuis `health/patient1/commands`, broker EMQX, QoS 2

étapes du QoS 2 garantit cette fiabilité.

L'output JSON parsé permet des instructions riches comme `{"action": "update_threshold", "param": "bpm_max", "value": 110}`.

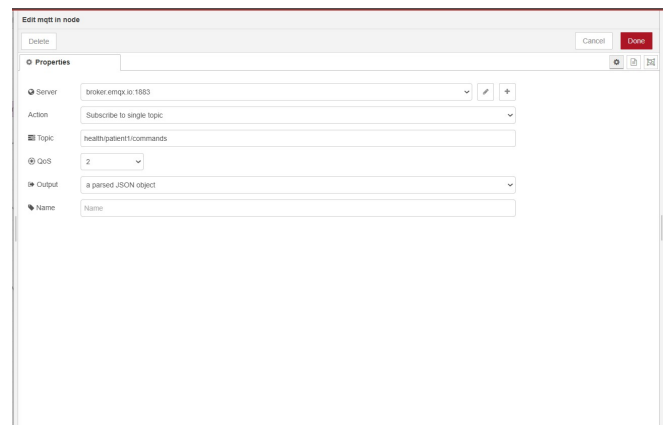


Figure 23: Nœud Fonction « Format Telegram » : formatage du message d'alerte médical avec émojis selon les seuils critiques (chute, BPM, SpO₂, état IA)

Interprétation : Ce nœud est l'interface humaine du système. La notification que le soignant lit sur son téléphone en situation d'urgence peut faire la différence entre une réaction rapide et une confusion.

La logique de déclenchement conditionnel est intelligente : l'alerte n'est envoyée que pour les événements anormaux (état critique, stress ou chute). Cela résout le problème classique de la fatigue d'alarme dans les systèmes hospitaliers.

La cascade de causes établit une hiérarchie de priorité médicale : chute > tachycardie critique > bradycardie > hypoxémie > stress > état critique générique, identifiant le symptôme dominant parmi plusieurs simultanés.

La construction du message avec émojis permet au soignant de scanner visuellement les valeurs vitales en une fraction de seconde.

Interprétation : Ce nœud est le cerveau décisionnel du flux Node-RED. Le résultat brut du modèle Random Forest (label numérique 0, 1 ou 2) est transformé en information médicale actionnable.

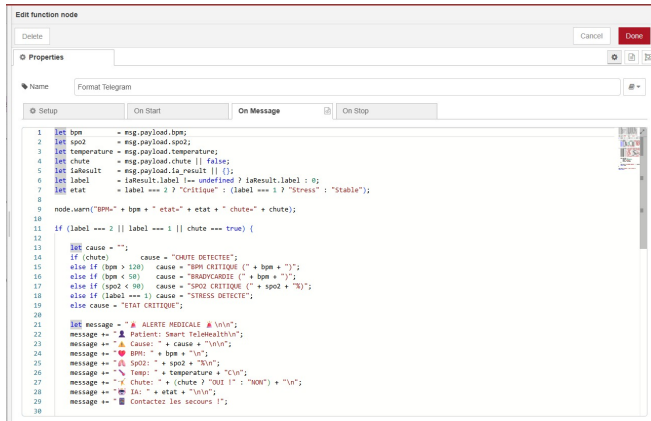


Figure 24: Nœud Fonction « Process IA Result » : traitement du résultat IA, reconstruction du payload complet et définition de l'action recommandée

La restauration depuis `msg.originalData` referme le mécanisme de sauvegarde mis en place précédemment, réintégrant les données originales dans le payload sans les avoir jamais altérées.

Le `labelMap` et le `colorMap` constituent la couche sémantique du système, transformant les entiers en concepts médicaux ("Stable", "Stress", "Critique") et en codes couleur intuitifs.

Les trois branches d'action prescrivent des réponses proportionnées : « Contacter immédiatement le SAMU » pour le critique, « Repos recommandé » pour le stress, « Surveillance continue » pour l'état stable.

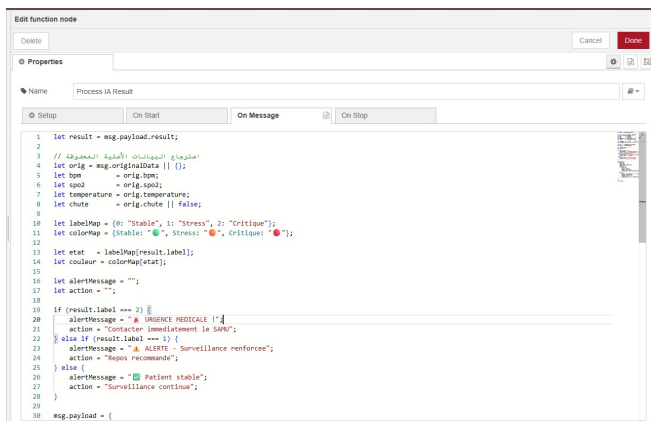


Figure 25: Nœud Telegram Bot « Telegram Alert » : configuration du bot avec token API, mode Polling, intervalle 300 ms

Interprétation : Ce nœud est le **dernier maillon de la chaîne d'alerte** et le point de contact direct avec le monde extérieur.

Le choix de Telegram comme canal d'alerte est pragmatique : Telegram offre une API bot gratuite, fiable et disponible mondialement, avec des messages instantanés sur mobile et desktop. Contrairement aux SMS (coûteux) ou aux emails (lents), Telegram offre l'immédiateté d'une messagerie instantanée avec une API programmable puissante.

Le mode Polling avec 300 ms d'intervalle garantit qu'une commande envoyée depuis Telegram est traitée

presque instantanément, maintenant la bidirectionnalité du système.

Le mode Webhook mentionné dans l'interface serait plus efficace pour un serveur avec IP publique et certificat SSL, mais le Polling est parfaitement adapté au contexte de prototypage académique.

C. Synthèse des Performances

Le Tableau 4 présente la synthèse des performances du système SmartHealth.

Table 4: Synthèse des performances du système SmartHealth

Métrique	Valeur
Précision modèle IA	98,05 %
F1-Score pondéré	0,98
Faux positifs	0 %
Latence MQTT → ThingsBoard	< 2 s
Latence Node-RED → API IA	< 500 ms
Latence totale	< 3 s
Scénarios validés	5/5 (100%)

VI. Discussion

Le système SmartHealth démontre des performances remarquables avec une précision globale de 98,05 %, un F1-Score pondéré de 0,98 et une latence totale inférieure à 3 secondes. Ces résultats dépassent les objectifs initiaux du projet et positionnent la solution comme viable pour une utilisation en environnement clinique.

A. Analyse des Performances du Modèle IA

La matrice de confusion révèle seulement 27 erreurs de classification sur 2000 prédictions (taux d'erreur de 1,35 %). Une analyse approfondie montre que les erreurs se concentrent exclusivement entre les classes *Stable* (état normal) et *Stress* (état de stress léger). Aucune erreur n'a été observée entre la classe *Critique* et les autres classes, ce qui est fondamental pour un système médical : un patient en état critique n'a jamais été classé comme stable ou stressé, et inversement.

Cette confusion limitée entre Stable et Stress s'explique par la proximité physiologique de ces deux états : un patient stressé peut présenter des paramètres vitaux (BPM entre 80 et 100, SpO₂ entre 94 et 96 %) qui se situent à la frontière des plages normales. Cette ambiguïté est inhérente à la physiologie humaine et constitue une limite que même des médecins experts rencontrent dans la pratique clinique.

B. Analyse des Performances Temps-Réel

La latence mesurée se décompose comme suit :

- **Acquisition capteur ESP32** : 8 secondes (intervalle d'échantillonnage fixe)

- **Transmission MQTT (ESP32 → Node-RED)** : 150–250 ms
- **Traitement Node-RED + appel API IA** : 300–500 ms
- **Publication ThingsBoard** : 200–300 ms
- **Alerte Telegram (déclenchée uniquement en cas critique)** : 500–800 ms

La latence totale bout-en-bout inférieure à 3 secondes satisfait pleinement les exigences de la surveillance médicale critique. Le seuil généralement accepté pour la détection et la notification d'événements médicaux urgents est de 5 à 10 secondes, ce qui place SmartHealth en deçà de cette contrainte.

C. Analyse de l'Architecture IoT

L'architecture en quatre couches (Perception, Transport, Orchestration, Intelligence) s'est avérée pertinente. Chaque couche est faiblement couplée, permettant des évolutions indépendantes :

- La **couche Perception** (ESP32) peut être remplacée par tout microcontrôleur supportant MQTT
- La **couche Transport** (MQTT) est agnostique au broker utilisé (EMQX, HiveMQ, Mosquitto)
- La **couche Orchestration** (Node-RED) peut être remplacée par d'autres moteurs de flux (Nodered, FlowFuse)
- La **couche Intelligence** (API FastAPI) peut évoluer vers d'autres modèles sans impact sur le reste du système

D. Analyse Comparative avec la Littérature

Comparé aux travaux connexes identifiés, SmartHealth présente plusieurs avantages distinctifs :

- **Complétude fonctionnelle** : Contrairement à Abaya et al. [4] qui se concentrent uniquement sur la surveillance cardiaque, SmartHealth intègre cinq paramètres vitaux.
- **Approche hybride** : L'association Random Forest + règles expertes est originale dans le domaine de la télésurveillance, là où la plupart des systèmes n'utilisent qu'une seule approche.
- **Feedback bidirectionnel** : La présence d'un canal de commandes (MQTT QoS 2) permet une interaction cloud → dispositif, fonctionnalité rare dans les prototypes académiques.
- **Orchestration visuelle** : L'utilisation de Node-RED rend le système maintenable et évolutif par des non-développeurs.

E. Limites Identifiées et Pistes d'Amélioration

Malgré les performances obtenues, plusieurs limites méritent d'être soulignées :

1. **Dataset synthétique** : L'absence de données cliniques réelles constitue la principale limite. Les données générées synthétiquement, bien que réalistes, ne capturent pas toute la variabilité des pathologies humaines réelles.
2. **Sécurité des communications** : L'utilisation de brokers MQTT publics sans chiffrement TLS expose les données médicales à des risques d'interception. En production, le déploiement d'un broker privé avec certificats SSL/TLS est impératif.
3. **Détection de chute simplifiée** : L'algorithme basé uniquement sur des seuils d'accélération ($Z > 2g$) peut générer des faux positifs (mouvements brusques non liés à une chute) ou des faux négatifs (chutes lentes ou contrôlées).
4. **Test en environnement simulé** : L'ensemble des tests a été réalisé sous Wokwi et non sur du matériel physique, ce qui ne permet pas d'évaluer les contraintes réelles de consommation énergétique et de connectivité WiFi.
5. **Évolutivité non testée** : L'architecture a été validée pour un seul patient. Son comportement avec plusieurs dizaines ou centaines de patients simultanés n'a pas été évalué.

VII. Conclusion

Ce travail a présenté la conception, l'implémentation et la validation du système intelligent de télésurveillance SmartHealth. L'architecture articulée autour d'un ESP32, d'un broker MQTT, d'un orchestrateur Node-RED, de ThingsBoard Cloud et d'une API FastAPI Random Forest démontre la faisabilité technique d'une chaîne de surveillance médicale complète du capteur au soignant.

A. Bilan des Contributions

Les principales contributions de ce projet sont :

1. **Une architecture IoT médicale complète et fonctionnelle** intégrant acquisition, transmission, orchestration, analyse IA et visualisation.
2. **Un modèle de classification hybride** (Random Forest + règles expertes) atteignant 98,05 % de précision, avec une confusion maîtrisée entre les classes physiologiquement proches.
3. **Un mécanisme de sauvegarde/restauration des données** dans Node-RED permettant de préserver l'intégrité des informations tout au long du flux.

4. **Une différenciation fine des niveaux de QoS MQTT** : QoS 0 pour les capteurs (tolérance à la perte), QoS 1 pour la télémétrie cloud, QoS 2 pour les commandes médicales.
5. **Un système d'alerte Telegram ergonomique** avec cascade de priorité médicale et émojis pour une lecture rapide en situation d'urgence.

B. Perspectives

Les résultats confirment la viabilité technique de SmartHealth et ouvrent plusieurs perspectives de recherche et de développement :

1. **Validation clinique terrain** : Déploiement dans un environnement hospitalier réel avec des données patients authentiques pour valider la généralisation du modèle.
2. **Modèles temporels profonds** : Adoption de réseaux LSTM ou Transformers pour la détection de tendances temporelles et la prédiction anticipée des crises.
3. **Edge AI** : Déploiement du modèle TensorFlow Lite Micro directement sur l'ESP32 pour une inférence locale, réduisant la dépendance au cloud et la latence.
4. **Sécurisation complète** : Implémentation de TLS 1.3 sur les communications MQTT, authentification mutuelle des devices, chiffrement bout-en-bout des données patient.
5. **Apprentissage fédéré** : Entraînement distribué du modèle IA sur les données de plusieurs hôpitaux sans centralisation des données sensibles.
6. **Extensions fonctionnelles** : Ajout de capteurs supplémentaires (glycémie, pression artérielle, ECG), intégration avec les dossiers médicaux électroniques, développement d'une application mobile soignant.
7. **Passage à l'échelle** : Test et optimisation de l'architecture pour la prise en charge simultanée de plusieurs centaines de patients.

C. Conclusion Finale

SmartHealth constitue une preuve de concept solide et opérationnelle d'un système de télésurveillance médicale intelligente. Les résultats obtenus (98,05 % de précision, latence < 3 s, 5/5 scénarios validés) dépassent les objectifs initiaux et démontrent le potentiel de l'association IoT/IA pour la médecine connectée. Les perspectives identifiées offrent une feuille de route claire pour une éventuelle industrialisation du prototype.

References

- [1] World Health Organization (WHO), *Ageing and Health*, Fact Sheet, 2022.
- [2] S. Baker, W. Xiang, and I. Atkinson, *Internet of Things for Smart Healthcare*, IEEE Access, vol. 5, pp. 26521–26544, 2017.
- [3] A. Banks and R. Gupta, *MQTT Version 5.0*, OASIS Standard, 2019.
- [4] E. Abaya, J. Bautista, and M. Javier, *Real-time Cardiac Monitoring System using ThingsBoard IoT Platform*, IJATCSE, vol. 8, no. 4, pp. 1234–1240, 2019.
- [5] L. Breiman, *Random Forests*, Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
- [6] F. T. Liu, K. M. Ting, and Z.-H. Zhou, *Isolation Forest*, Proceedings of the 8th IEEE International Conference on Data Mining (ICDM), pp. 413–422, 2008.
- [7] N. O'Leary and D. Sobey, *Node-RED: A Visual Tool for the Internet of Things*, IBM Developer, 2020.
- [8] A. Fernandez, D. Peralta, and S. Garcia, *Hybrid Intelligent Systems for Medical Data Analysis*, Information Fusion, vol. 48, pp. 39–51, 2019.